



Python Script Writing Cheat Sheet

How to Write a Script From Scratch — Rules, Structure & Common Mistakes

STRUCTURE — How Every Script Is Laid Out

Follow this order every time you write a script:

```
# STEP 1 – Imports (grab your toolboxes first)
import requests
import csv

# STEP 2 – Variables (set up your data)
name = "Vickie"
users = ["Jordan", "Maya", "Chris"]

# STEP 3 – Functions (define your recipes)
def greet(person):
    print(f"Hello, {person}!")

# STEP 4 – Your actual logic (run the thing)
for user in users:
    greet(user)
```

■ Think of it like cooking: gather ingredients (imports/variables), write the recipe (functions), then cook (run the logic).

PUNCTUATION RULES — What You Need & What You Don't

Symbol	Used For	Required?	Example
:	After if / else / for / while / def	■ YES	if age > 18:
()	After function names	■ YES	print("hi")
" " or ' '	Around text/strings	■ YES	name = "Vickie"
#	Comments (notes to yourself)	Optional	# this is a note
;	End of line	■ NOT needed	Never use this
{ }	Code blocks	■ NOT needed	Python uses indent instead
f"..."	String with variable inside	Optional but useful	f"Hi {name}"
[]	Lists	When making a list	fruits = ["apple"]
{ : }	Dictionaries	When making a dict	{"name": "Vickie"}

INDENTATION — The #1 Rule in Python

Python uses spacing (indent) instead of { } to group code. Everything inside an if, loop, or function must be indented.

■ CORRECT

```
if age > 18:
    print("adult") # 4 spaces in
    print("can vote") # still inside the if
print("this is outside") # back to normal
```

■ WRONG — will break

```
if age > 18:
print("adult") # no indent = Python freaks out
```

■ ■ Use 4 spaces OR 1 tab — but never mix them in the same file. Pick one and stick with it.

STRINGS vs NUMBERS — Quotes or No Quotes?

Simple rule: words/text = quotes. Numbers = no quotes.

Type	Quotes?	Example	Notes
Text (string)	■ YES	name = "Vickie"	Single or double quotes both work
Number (integer)	■ NO	age = 30	No quotes, no decimal
Decimal (float)	■ NO	weight = 145.5	Has a decimal point
True / False	■ NO	active = True	Capital T and F — always
Nothing / empty	■ NO	value = None	Like null in other languages

THE COLON RULE — When to Use :

Only 5 situations need a colon. Memorize these:

Keyword	What it does	Example
if / elif / else	Makes a decision	if name == "Vickie":
for	Loops through a list	for user in users:
while	Loops while something is true	while count < 10:
def	Defines a function	def my_function():
try / except	Handles errors gracefully	try: / except:

COMMON BEGINNER MISTAKES

Forgetting the colon

```
if age > 18 # ■ missing colon
if age > 18: # ■ correct
```

Mixing up = and ==

```
name = "Vickie" # = means ASSIGN (store a value)
if name == "Vickie": # == means COMPARE (is it equal?)
```

Quotes mismatch

```
name = "Vickie' # ■ started with " ended with '
name = "Vickie" # ■ match your quotes
```

Forgetting to indent after a colon

```
def greet():
print("hi") # ■ not indented

def greet():
    print("hi") # ■ indented 4 spaces
```

Treating a number as a string

```
age = "30" # ■ this is TEXT "30", not the number 30
age = 30 # ■ this is the number
```

BEFORE YOU RUN — Quick Checklist

Check	Ask Yourself
Imports at top?	Did I import everything I need before using it?
Colons in place?	Every if / for / while / def ends with : ?
Indentation consistent?	Everything inside a block indented the same way?
Quotes matched?	Every " has a closing " — no mixing with ' ?
Variables named?	Did I actually create the variable before using it?
= vs == ?	Assigning a value (=) vs comparing (==)?
Print to test?	Added print() statements to see what's happening?

■ *Pro tip: When something breaks, add print() everywhere to see what the computer is actually seeing. It's the fastest way to debug.*

COPY THIS — Blank Script Starter Template

Every time you start a new script, use this as your blank canvas:

```
# ■■ IMPORTS ■■
# import whatever you need here

# ■■ VARIABLES ■■
# set up your data here

# ■■ FUNCTIONS ■■
def my_function():
    pass # "pass" is a placeholder – replace with real code

# ■■ MAIN LOGIC ■■
# write what actually runs here
my_function()
```